

PHYSICAL DATABASE DESIGN AND QUERY PROCESSING

Module 4
BCSE302L – Database
Systems

AGENDA

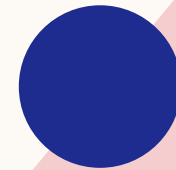
Basic Concepts

Ordered Indices

B⁺-Tree Index Files

B-Tree Index Files

Hashing





QUERY LANGUAGES

- Language in which user requests information from the database.
- Categories of languages
 - procedural
 - non-procedural
- “Pure” languages:
 - Relational Algebra
 - Tuple Relational Calculus
 - Domain Relational Calculus
- Pure languages form underlying basis of query languages that people use.



RELATIONAL ALGEBRA

- Procedural language
- Six basic operators
 - select
 - project
 - union
 - set difference
 - Cartesian product
 - rename
- The operators take one or more relations as inputs and give a new relation as a result.

SELECT OPERATION – EXAMPLE

- Relation r

A	B	C	D
α	α	1	7
α	β	5	7
β	β	12	3
β	β	23	10

$$\sigma_{A=B \wedge D > 5}(r)$$

A	B	C	D
α	α	1	7
β	β	23	10

SELECT OPERATION

- Notation: $\sigma_p(r)$
- p is called the **selection predicate**
- Defined as:

$$\sigma_p(r) = \{t \mid t \in r \text{ and } p(t)\}$$

- Where p is a formula in propositional calculus consisting of **terms** connected by :
 $\wedge$ (**and**), $\vee$ (**or**), $\neg$ (**not**). Each **term** is one of:
 $\langle \text{attribute} \rangle \text{ op } \langle \text{attribute} \rangle$ or $\langle \text{constant} \rangle$
- where op is one of: $=, \neq, >, \geq, <, \leq$

SELECT OPERATION

- Find the tuples of the loan relation where the branch is “SBI”

$$\sigma_{branch-name = \text{“SBI”}}(loan)$$

- Find the tuples in which the amount lent is more than 1200

$$\sigma_{amount > 1200}(loan)$$

- Find those tuples pertaining to loans of more than 1200 made by the SBI branch

$$\sigma_{branch-name = \text{“SBI”} \wedge amount > 1200}(loan)$$

PROJECT OPERATION – EXAMPLE

- Relation r :

A	B	C
α	10	1
α	20	1
β	30	1
β	40	2

$\Pi_{A,C}(r)$

A	C
α	1
α	1
β	1
β	2

=

A	C
α	1
β	1
β	2

PROJECT OPERATION

- Notation: $\Pi_{A_1, A_2, A_3, \dots, A_k}(r)$
where A_1, A_2 are attribute names and r is a relation name.
- The result is defined as the relation of k columns obtained by erasing the columns that are not listed
- Duplicate rows removed from result, since relations are sets
-

COMPOSITION OF RELATIONAL OPERATIONS

- List all loan numbers and the amount of the loan

$$\Pi_{loan-no, amount}(loan)$$

- Find customers who live in Mumbai

$$\Pi_{cusomter-name} \left(\sigma_{customer-city="mumbai"}(cusomter) \right)$$

UNION OPERATION – EXAMPLE

- Relations r, s :

A	B
α	1
α	2
β	1

r

A	B
α	2
β	3

s

$r \cup s$

A	B
α	1
α	2
β	1
β	3

UNION OPERATION

- Notation: $r \cup s$
- Defined as:

$$r \cup s = \{t \mid t \in r \text{ or } t \in s\}$$

- For $r \cup s$ to be valid.
 - r, s must have the *same arity* (same number of attributes)
 - The attribute domains must be *compatible* (e.g., 2nd column of r deals with the same type of values as does the 2nd column of s)

UNION OPERATIONS

- Find the names of all bank customers who have either an account or loan or both.

$$\begin{array}{c} \Pi_{customer-name}(depositor) \\ \cup \\ \Pi_{customer-name}(borrower) \end{array}$$

SET DIFFERENCE OPERATION – EXAMPLE

- Relations r, s :

A	B
α	1
α	2
β	1

r

A	B
α	2
β	3

s

$r - s$

A	B
α	1
β	1

SET DIFFERENCE OPERATION

- Notation: $r - s$
- Defined as:

$$r - s = \{t | t \in r \text{ or } t \notin s\}$$

- Set differences must be taken between *compatible* relations.
 - r and s must have the *same arity*
 - attribute domains of r and s must be compatible

UNION OPERATIONS

- Find all customers of the bank who have a loan but not an account

$$\begin{array}{c} \Pi_{customer-name}(depositor) \\ - \\ \Pi_{customer-name}(borrower) \end{array}$$

CARTESIAN PRODUCT OPERATION – EXAMPLE

- Relations r, s :

A	B
-----	-----

α	1
β	2

r

C	D	E
-----	-----	-----

α	10	a
β	10	a
β	20	b
γ	10	b

s

$r \times s$

A	B	C	D	E
-----	-----	-----	-----	-----

α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b

CARTESIAN-PRODUCT OPERATION

- Notation: $r \times s$
- Defined as:
$$r \times s = \{tq | t \in r \text{ and } q \in s\}$$
- Assume that attributes of $r(R)$ and $s(S)$ are disjoint.
(That is, $R \cap S = \phi$).
- If attributes of $r(R)$ and $s(S)$ are not disjoint, then renaming must be used.

COMPOSITION OF OPERATIONS

- Can build expressions using multiple operations
- Example:

- $r \times s$

A	B	C	D	E
α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b

$$\sigma_{A=C}(r \times s)$$

A	B	C	D	E
α	1	α	10	a
β	2	β	20	a
β	2	β	20	b

CARTESIAN PRODUCT

- Find names of all customers who have a loan at the AXIS bank

$$\Pi_{customer-name} (\sigma_{branch-name="axis" \wedge borrower.loan-no=loan.loan-no}(borrower \times loan))$$

RENAME OPERATION

- Allows us to name, and therefore to refer to, the results of relational-algebra expressions.
- Allows us to refer to a relation by more than one name.

Example:

$$\rho_x(E)$$

returns the expression E under the name X

- If a relational-algebra expression E has arity n , then

$$\rho_x(A_1, A_2, \dots, A_n)(E)$$

returns the result of expression E under the name X , and with the attributes renamed to $A_1, A_2, \dots, A_n$.

RENAME OPERATIONS

- Find largest account balance in the bank

$$\frac{\Pi_{balance}(account)}{\Pi_{account.balance}} - \left(\sigma_{account.balance < d.balance}(account \times \rho_d(account)) \right)$$

RENAME OPERATIONS

- Find the names of all customers who live on the same street and in the same city as Shruthi

$$\begin{aligned} & \Pi_{customer.customer-name} \\ & \left(\sigma_{customer.street = sh.st \wedge customer.city = sh.city} \right. \\ & \quad \left(customer \times \rho_{sh}(st, city) \right. \\ & \quad \quad \left(\Pi_{street, city} \left(\right. \right. \\ & \quad \quad \quad \left. \sigma_{customer-name = "shruthi"}(customer) \right) \left. \right) \left. \right) \end{aligned}$$

EXAMPLE

- Find the names of all customer who have a loan at the AXIS branch but do not have an account at any branch of the bank

$$\begin{aligned} & \Pi_{customer.customer-name} \\ & \quad (\sigma_{branch-name="axis"} \\ & \quad (\sigma_{borrower.loan-no=loan.loan-no}(borrower \\ & \quad \times loan))) \\ & \quad - \\ & \quad (\Pi_{customer-name}(depositor)) \end{aligned}$$

EXAMPLE

- Find the names of all customer who have both a loan and an account.

$$\Pi_{customer-name}(borrower) \cap \Pi_{customer-name}(depositor)$$

FORMAL DEFINITION

- A basic expression in the relational algebra consists of either one of the following:
 - A relation in the database
 - A constant relation
- Let E_1 and E_2 be relational-algebra expressions; the following are all relational-algebra expressions:
 - $E_1 \cup E_2$
 - $E_1 - E_2$
 - $E_1 \times E_2$
 - $\sigma_P(E_1)$, P is a predicate on attributes in E_1
 - $\Pi_S(E_1)$, S is a list consisting of some of the attributes in E_1
 - $\rho_x(E_1)$, x is the new name for the result of E_1



ADDITIONAL OPERATIONS

- We define additional operations that do not add any power to the relational algebra, but that simplify common queries.
 - Set intersection
 - Natural join
 - Division
 - Assignment

SET-INTERSECTION OPERATION

- Notation: $r \cap s$
- Defined as:

$$r \cap s = \{ t \mid t \in r \text{ and } t \in s \}$$

- Assume:
 - r, s have the *same arity*
 - attributes of r and s are compatible
- Note: $r \cap s = r - (r - s)$

SET-INTERSECTION OPERATION - EXAMPLE

- Relation r, s :

A	B
α	1
α	2
β	1

r

A	B
α	2
β	3

s

- $r \cap s$

A	B
α	2

NATURAL-JOIN OPERATION

- Notation: $r \bowtie s$
- Let r and s be relations on schemas R and S respectively. Then, $r \bowtie s$ is a relation on schema $R \cup S$ obtained as follows:
 - Consider each pair of tuples t_r from r and t_s from s .
 - If t_r and t_s have the same value on each of the attributes in $R \cap S$, add a tuple t to the result, where
 - t has the same value as t_r on r
 - t has the same value as t_s on s

NATURAL-JOIN OPERATION

- Example:

$$R = (A, B, C, D)$$

$$S = (E, B, D)$$

- Result schema = (A, B, C, D, E)
- $r \bowtie s$ is defined as:

$$\Pi_{r.A, r.B, r.C, r.D, s.E} (\sigma_{r.B = s.B \wedge r.D = s.D} (r \times s))$$

NATURAL JOIN OPERATION – EXAMPLE

- Relations r , s :

A	B	C	D
α	1	α	a
β	2	γ	a
γ	4	β	b
α	1	γ	a
δ	2	β	b

r

B	D	E
1	a	α
3	a	β
1	a	γ
2	b	δ
3	b	ϵ

s

$r \bowtie s$

A	B	C	D	E
α	1	α	a	α
α	1	α	a	γ
α	1	γ	a	α
α	1	γ	a	γ
δ	2	β	b	δ

DIVISION OPERATION

- Notation: $r \div s$
- Suited to queries that include the phrase “for all”.
- Let r and s be relations on schemas R and S respectively where $R = (A_1, \dots, A_m, B_1, \dots, B_n)$, $S = (B_1, \dots, B_n)$

The result of $r \div s$ is a relation on schema

$$R - S = (A_1, \dots, A_m)$$

$$r \div s = \{ t \mid t \in \Pi_{R-S}(r) \wedge \forall u \in s (tu \in r) \}$$

DIVISION OPERATION – EXAMPLE

- Relation r, s : $r \div s$

A	B
α	1
α	2
α	3
β	1
γ	1
δ	1
δ	3
δ	4
ϵ	6
ϵ	1
β	2

r

B
1
2

s

A
α
β

- Relation r, s : $r \div s$

A	B	C	D	E
α	a	α	a	1
α	a	γ	a	1
α	a	γ	b	1
β	a	γ	a	1
β	a	γ	b	3
γ	a	γ	a	1
γ	a	γ	b	1
γ	a	β	b	1

r

D	E
a	1
b	1

s

A	B	C
α	a	γ
γ	a	γ

DIVISION OPERATION (CONT.)

- Property
 - Let $q - r \div s$
 - Then q is the largest relation satisfying $q \times s \subseteq r$
- Definition in terms of the basic algebra operation
Let $r(R)$ and $s(S)$ be relations, and let $S \subseteq R$
$$r \div s = \Pi_{R-S}(r) - \Pi_{R-S}((\Pi_{R-S}(r) \times s) - \Pi_{R-S,S}(r))$$
- To see why
 - $\Pi_{R-S,S}(r)$ simply reorders attributes of r
 - $\Pi_{R-S}(\Pi_{R-S}(r) \times s) - \Pi_{R-S,S}(r)$ gives those tuples t in $\Pi_{R-S}(r)$ such that for some tuple $u \in s$, $tu \notin r$.

ASSIGNMENT OPERATION

- The assignment operation ($\leftarrow$) provides a convenient way to express complex queries.
- Write query as a sequential program consisting of
 - a series of assignments
 - followed by an expression whose value is displayed as a result of the query.
- Assignment must always be made to a temporary relation variable.

ASSIGNMENT OPERATION

- Example: Write $r \div s$ as

$$temp1 \leftarrow \Pi_{R-S}(r)$$

$$temp2 \leftarrow \Pi_{R-S}((temp1 \times s) - \Pi_{R-S,S}(r))$$

$$result = temp1 - temp2$$

- The result to the right of the $\leftarrow$ is assigned to the relation variable on the left of the $\leftarrow$.
- May use variable in subsequent expressions.

EXAMPLE QUERIES

- Find all customers who have an account at all the branches located in Vellore.

$$Q1 \leftarrow \Pi_{branch-name} \left(\sigma_{branch-city="vellore"}(branch) \right)$$

$$Q2 \leftarrow \Pi_{customer-name, branch-name}(depositor \bowtie account)$$

$$Q2 \div Q1$$



EXTENDED RELATIONAL- ALGEBRA-OPERATIONS

- Generalized Projection
- Outer Join
- Aggregate Functions

GENERALIZED PROJECTION

- Extends the projection operation by allowing arithmetic functions to be used in the projection list.

$$\Pi_{F_1, F_2, \dots, F_n}(E)$$

- E is any relational-algebra expression
- Each of $F_1, F_2, \dots, F_n$ are arithmetic expressions involving constants and attributes in the schema of E .
- Given relation *credit-info(customer-name, limit, credit-balance)*, find how much more each person can spend:

$$\Pi_{customer-name, limit - credit-balance \text{ as } credit-available}(credit-info)$$

AGGREGATE FUNCTIONS AND OPERATIONS

- **Aggregation function** takes a collection of values and returns a single value as a result.
- **avg**: average value, **min**: minimum value, **max**: maximum value, **sum**: sum of values, **count**: number of values
- **Aggregate operation** in relational algebra

$$G_1, G_2, \dots, G_n \mathcal{G} F_1(A_1), F_2(A_2), \dots, F_n(A_n) (E)$$

- E is any relational-algebra expression
- $G_1, G_2, \dots, G_n$ is a list of attributes on which to group (can be empty)
- Each F_i is an aggregate function
- Each A_i is an attribute name

AGGREGATE OPERATION – EXAMPLE

- Relation r : $\mathcal{G}_{sum(c)}(r)$

A	B	C
α	α	7
α	β	7
β	β	3
β	β	10

$sum-C$

27

Relation $account$

$branch-name$	$account-number$	$balance$
sbi	A-102	400
sbi	A-201	900
Brighton	A-217	750
Brighton	A-215	750
Redwood	A-222	700

$branch - name \mathcal{G}_{sum(balance)}(account)$

$branch-name$	$balance$
sbi	1300
Brighton	1500
Redwood	700

AGGREGATE FUNCTIONS (CONT.)

- Result of aggregation does not have a name
 - Can use rename operation to give it a name
 - For convenience, we permit renaming as part of aggregate operation

branch — name $g_{sum(balance)}(account)$

OUTER JOIN

- An extension of the join operation that avoids loss of information.
- Computes the join and then adds tuples from one relation that do not match tuples in the other relation to the result of the join.
- Uses *null* values:
 - *null* signifies that the value is unknown or does not exist
 - All comparisons involving *null* are (roughly speaking) **false** by definition.

OUTER JOIN – EXAMPLE

- Relation *loan*

loan-number	branch-name	amount
L-170	Downtown	3000
L-230	Redwood	4000
L-260	sbi	1700

- Relation *borrower*

customer-name	loan-number
Jones	L-170
Smith	L-230
Hayes	L-155

OUTER JOIN – EXAMPLE

- Inner Join

loan ⋈ *Borrower*

loan-number	branch-name	amount	customer-name
L-170	Downtown	3000	Jones
L-230	Redwood	4000	Smith

- Left Outer Join

loan ⋈_L *Borrower*

loan-number	branch-name	amount	customer-name
L-170	Downtown	3000	Jones
L-230	Redwood	4000	Smith
L-260	sbi	1700	null

OUTER JOIN – EXAMPLE

- **Right Outer Join**

loan ⋈_⊃ *Borrower*

loan-number	branch-name	amount	customer-name
L-170	Downtown	3000	Jones
L-230	Redwood	4000	Smith
L-155	null	null	Hayes

- **Full Outer Join**

loan ⋈_{⊃⊃} *Borrower*

loan-number	branch-name	amount	customer-name
L-170	Downtown	3000	Jones
L-230	Redwood	4000	Smith
L-260	sbi	1700	null
L-155	null	null	Hayes

NULL VALUES

- It is possible for tuples to have a null value, denoted by *null*, for some of their attributes
- *null* signifies an unknown value or that a value does not exist.
- The result of any arithmetic expression involving *null* is *null*.
- Aggregate functions simply ignore null values
 - Is an arbitrary decision. Could have returned null as result instead.
 - We follow the semantics of SQL in its handling of null values
- For duplicate elimination and grouping, null is treated like any other value, and two nulls are assumed to be the same
 - Alternative: assume each null is different from each other
 - Both are arbitrary decisions, so we simply follow SQL

NULL VALUES

- Comparisons with null values return the special truth value *unknown*
 - If *false* was used instead of *unknown*, then $\text{not } (A < 5)$ would not be equivalent to $A \geq 5$
- Three-valued logic using the truth value *unknown*:
 - OR: $(\text{unknown or true}) = \text{true}$,
 $(\text{unknown or false}) = \text{unknown}$
 $(\text{unknown or unknown}) = \text{unknown}$
 - AND: $(\text{true and unknown}) = \text{unknown}$,
 $(\text{false and unknown}) = \text{false}$,
 $(\text{unknown and unknown}) = \text{unknown}$
 - NOT: $(\text{not unknown}) = \text{unknown}$
 - In SQL “*P* is **unknown**” evaluates to true if predicate *P* evaluates to *unknown*
- Result of select predicate is treated as *false* if it evaluates to *unknown*



MODIFICATION OF THE DATABASE

- The content of the database may be modified using the following operations:
 - Deletion
 - Insertion
 - Updating
- All these operations are expressed using the assignment operator.

DELETION

- A delete request is expressed similarly to a query, except instead of displaying tuples to the user, the selected tuples are removed from the database.
- Can delete only whole tuples; cannot delete values on only particular attributes
- A deletion is expressed in relational algebra by:
 - $r \leftarrow r - E$
- where r is a relation and E is a relational algebra query.

DELETION EXAMPLES

- Delete all account records in the SBI branch.

$account \leftarrow account - \sigma_{branch-name = "sbi"}(account)$

- Delete all loan records with amount in the range of 0 to 50

$loan \leftarrow loan - \sigma_{amount \geq 0 \text{ and } amount \leq 50}(loan)$

- Delete all accounts at branches located in Needham.

$r_1 \leftarrow \sigma_{branch-city = "Needham"}(account \bowtie branch)$

$r_2 \leftarrow \Pi_{branch-name, account-number, balance}(r_1)$

$r_3 \leftarrow \Pi_{customer-name, account-number}(r_2 \bowtie depositor)$

$account \leftarrow account - r_2$

$depositor \leftarrow depositor - r_3$

INSERTION

- To insert data into a relation, we either:
 - specify a tuple to be inserted
 - write a query whose result is a set of tuples to be inserted
- in relational algebra, an insertion is expressed by:
 - $$r \leftarrow r \cup E$$
 - where r is a relation and E is a relational algebra expression.
- The insertion of a single tuple is expressed by letting E be a constant relation containing one tuple.

INSERTION EXAMPLES

- Insert information in the database specifying that Smith has \$1200 in account A-973 at the sbi branch.

$account \leftarrow account \cup \{("sbi", A-973, 1200)\}$

$depositor \leftarrow depositor \cup \{("Smith", A-973)\}$

- Provide as a gift for all loan customers in the SBI branch, a \$200 savings account. Let the loan number serve as the account number for the new savings account.

$r_1 \leftarrow (\sigma_{branch-name = "sbi"}(borrower \quad loan))$

$account \leftarrow account \cup \Pi_{branch-name, account-number, 200}(r_1)$

$depositor \leftarrow depositor \cup \Pi_{customer-name, loan-number}(r_1)$

UPDATING

- A mechanism to change a value in a tuple without changing *all* values in the tuple
- Use the generalized projection operator to do this task
- $$r \leftarrow \Pi_{F_1, F_2, \dots, F_I}(r)$$
- Each F_i is either
 - the i th attribute of r , if the i th attribute is not updated, or,
 - if the attribute is to be updated F_i is an expression, involving only constants and the attributes of r , which gives the new value for the attribute

UPDATE EXAMPLES

- Make interest payments by increasing all balances by 5 percent.

$$account \leftarrow \Pi_{AN, BN, BAL} * 1.05 (account)$$

- Pay all accounts with balances over \$10,000 6 percent interest and pay all others 5 percent

$$account \leftarrow \Pi_{AN, BN, BAL} * 1.06 (\sigma_{BAL > 10000} (account)) \\ \cup \Pi_{AN, BN, BAL} * 1.05 (\sigma_{BAL \leq 10000} (account))$$

VIEWS

- In some cases, it is not desirable for all users to see the entire logical model (i.e., all the actual relations stored in the database.)
- Consider a person who needs to know a customer's loan number but has no need to see the loan amount. This person should see a relation described, in the relational algebra, by

$$\Pi_{customer-name, loan-number} (borrower \bowtie loan)$$

- Any relation that is not of the conceptual model but is made visible to a user as a “virtual relation” is called a **view**.

VIEW DEFINITION

- A view is defined using the **create view** statement which has the form
create view v as <query expression>
- where **<query expression>** is any legal relational algebra query expression. The view name is represented by v .
- Once a view is defined, the view name can be used to refer to the virtual relation that the view generates.
- View definition is not the same as creating a new relation by evaluating the query expression
 - Rather, a view definition causes the saving of an expression; the expression is substituted into queries using the view.

VIEW EXAMPLES

- Consider the view (named *all-customer*) consisting of branches and their customers.

create view *all-customer* as

$\Pi_{branch-name, customer-name} (depositor \bowtie account)$
 $\cup \Pi_{branch-name, customer-name} (borrower \bowtie loan)$

- We can find all customers of the sbi branch by writing:

$\Pi_{customer-name} (\sigma_{branch-name = \text{"sbi"}} (all-customer))$

UPDATES THROUGH VIEW

- Database modifications expressed as views must be translated to modifications of the actual relations in the database.
- Consider the person who needs to see all loan data in the *loan* relation except *amount*. The view given to the person, *branch-loan*, is defined as:
 - **create view *branch-loan* as**
 - $\Pi_{branch-name, loan-number}(loan)$
- Since we allow a view name to appear wherever a relation name is allowed, the person may write:
 - $branch-loan \leftarrow branch-loan \cup \{("sbi", L-37)\}$
 -

UPDATES THROUGH VIEWS

- The previous insertion must be represented by an insertion into the actual relation *loan* from which the view *branch-loan* is constructed.
- An insertion into *loan* requires a value for *amount*. The insertion can be dealt with by either.
 - rejecting the insertion and returning an error message to the user.
 - inserting a tuple (“L-37”, “sbi”, *null*) into the *loan* relation
- Some updates through views are impossible to translate into database relation updates
 - create view *v* as $\sigma_{branch-name = \text{“sbi”}}(account)$
 $v \leftarrow v \cup (L-99, \text{Downtown}, 23)$
- Others cannot be translated uniquely
 - $all-customer \leftarrow all-customer \cup \{(\text{“sbi”}, \text{“John”})\}$
 - Have to choose loan or account, and create a new loan/account number!

UPDATES THROUGH VIEWS

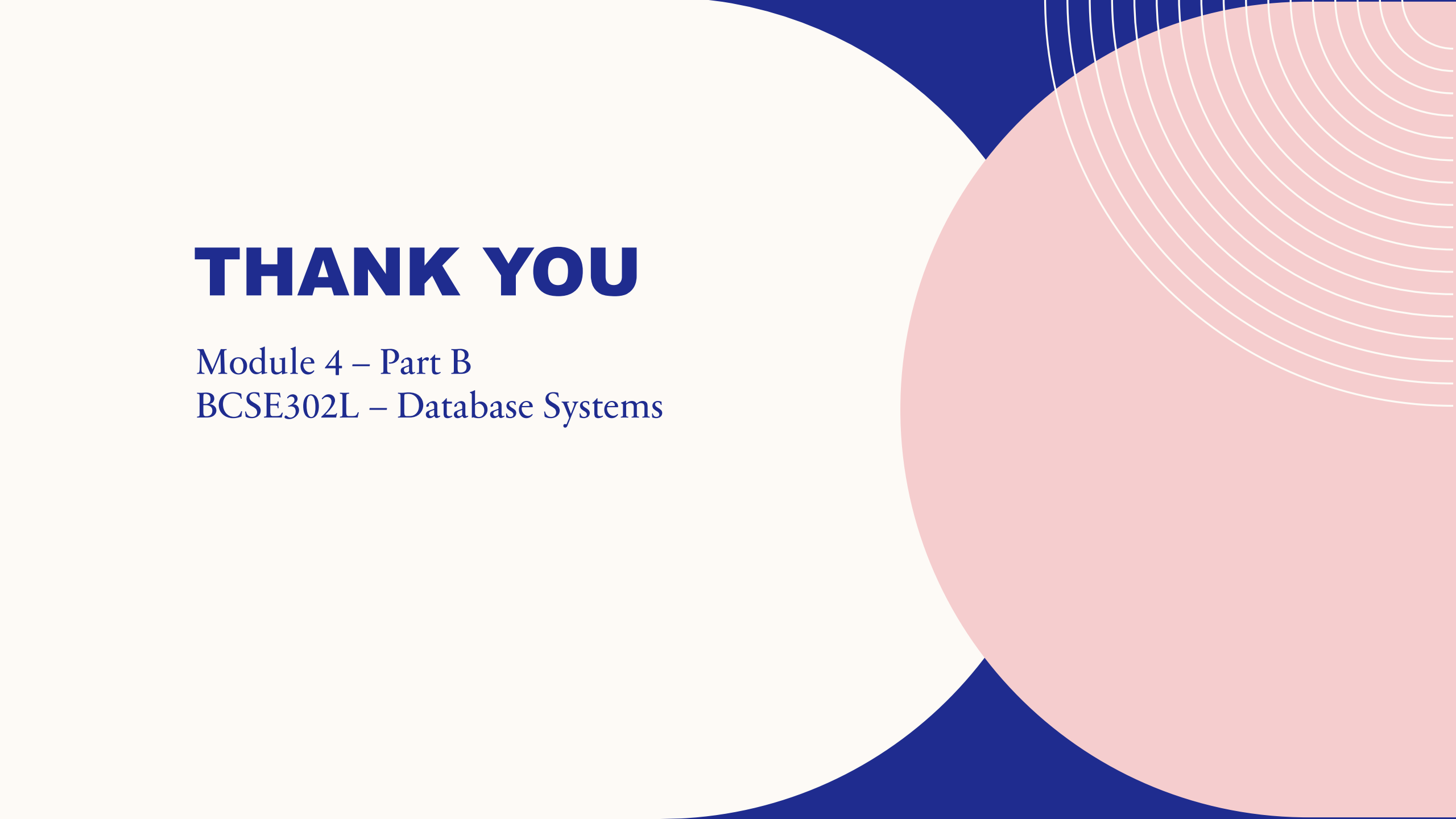
- One view may be used in the expression defining another view
- A view relation v_1 is said to *depend directly* on a view relation v_2 if v_2 is used in the expression defining v_1
- A view relation v_1 is said to *depend on* view relation v_2 if either v_1 depends directly to v_2 or there is a path of dependencies from v_1 to v_2
- A view relation v is said to be *recursive* if it depends on itself.

VIEWS DEFINED USING OTHER VIEWS

- One view may be used in the expression defining another view
- A view relation v_1 is said to *depend directly* on a view relation v_2 if v_2 is used in the expression defining v_1
- A view relation v_1 is said to *depend on* view relation v_2 if either v_1 depends directly to v_2 or there is a path of dependencies from v_1 to v_2
- A view relation v is said to be *recursive* if it depends on itself.

VIEW EXPANSION

- A way to define the meaning of views defined in terms of other views.
- Let view v_1 be defined by an expression e_1 that may itself contain uses of view relations.
- View expansion of an expression repeats the following replacement step:
- **repeat**
 - Find any view relation v_i in e_1
 - Replace the view relation v_i by the expression defining v_i
- **until** no more view relations are present in e_1
- As long as the view definitions are not recursive, this loop will terminate



THANK YOU

Module 4 – Part B
BCSE302L – Database Systems